

DNS cache poisoning and resolver hardening

Follow-along: play the attacker, then the defender

Overview

Definition: Caching resolver, and authoritative server

Two different jobs, done by two different machines. An *authoritative server* holds the records for a set of names and answers questions about them from its own zone file; it is where the data lives. A *caching resolver* holds no records of its own. When a client asks it for a name, it asks the authoritative server, hands the reply back to the client, and keeps a copy so the next client asking the same question is answered without a second query going out. Every host on a campus network is usually configured with the address of one caching resolver and asks it for everything.

Definition: Cache poisoning

Getting a caching resolver to store a record that its authoritative server never sent, so that every client the resolver serves is given the attacker's answer instead of the real one. Nothing on the client is touched, and nothing on the authoritative server is touched. The client asks the question it always asks, the resolver answers from its cache as it always does, and the answer is wrong.

A **caching resolver** answers a client from memory whenever it can, and goes and asks an authoritative server whenever it cannot. That second case is the one an attacker cares about, because it puts a question on the wire and leaves the resolver waiting for a UDP datagram to come back. **Cache poisoning** is arriving with that datagram first. If a forged answer reaches the resolver before the real one, and it matches what the resolver was waiting for, the resolver stores it and serves it to every client that asks, for as long as the forged answer says to, up to the resolver's own ceiling on how long anything may be cached.

This lab runs that attack from a machine that cannot see the query it is answering, then closes it three times over. The attacker sits on a network the resolver's traffic never crosses, so it has to guess what it cannot read; you will measure exactly how much guessing that is, and watch the guess succeed. The defences are then a resolver setting each: choosing a fresh source port for every query, refusing to look names up for clients outside the campus, and checking the zone's signatures before believing an answer. Every query and answer in this lab travels over UDP to port 53; DNS runs over TCP on port 53 as well, and no step here uses it. DNS carried inside TLS or HTTPS, an attacker positioned on the campus segment where it can read the query rather than guess at it, and DNS used as a channel to smuggle data out of a network, are each a separate lab and none of them is touched here.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

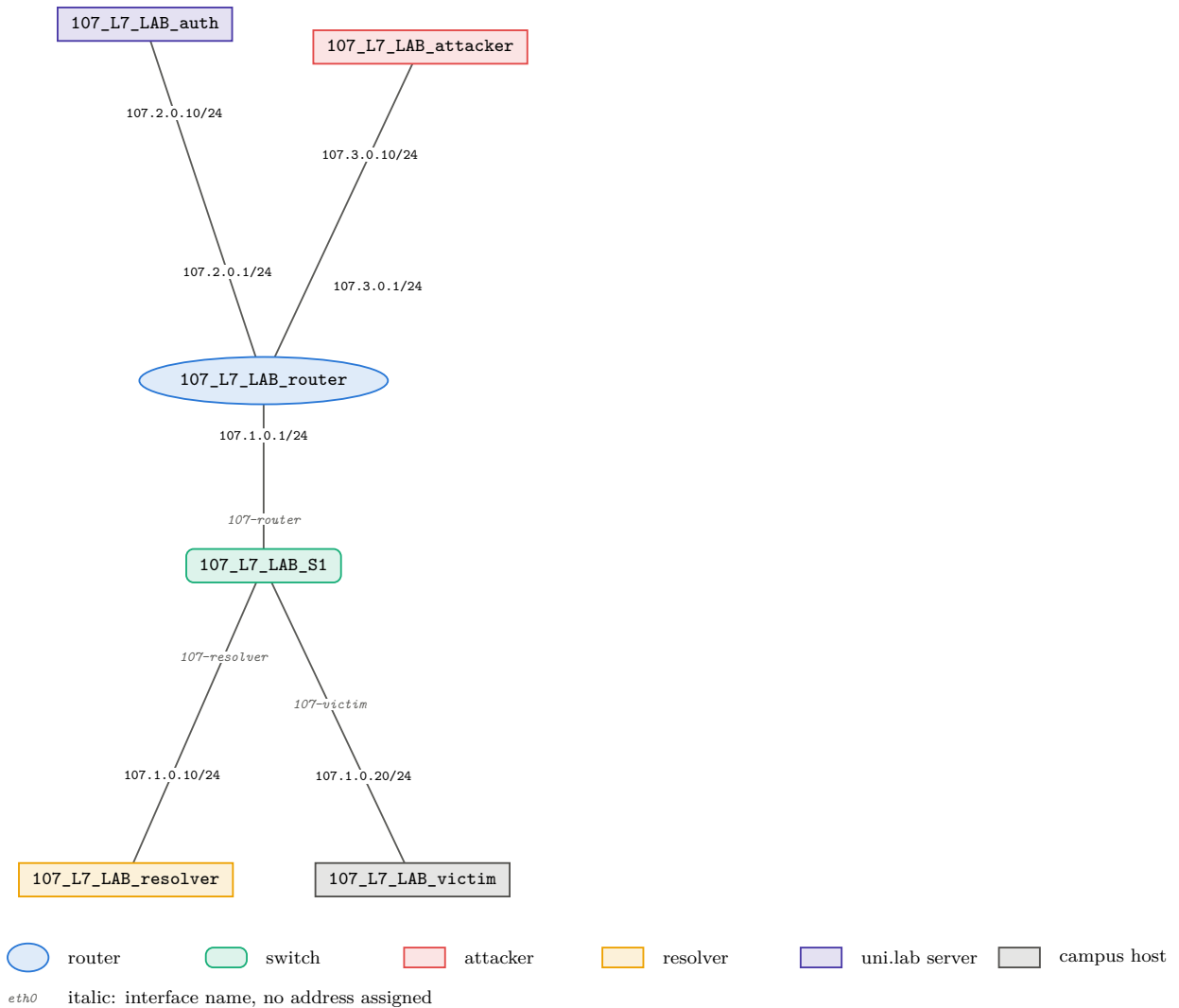
The root menu lists one category per row. Select **Application layer attacks and mitigations**, then **DNS cache poisoning and resolver hardening**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: which address the campus is currently given for the name, and which machine actually replies there. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (**attacker**, **victim**, **resolver**, **auth**, **router**) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** stops the attack and drops your defence, returning the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

A campus network with one caching resolver on it, a host that uses it, and the authoritative server for the name that host is looking up. The attacker is on a third network, one router hop from the resolver and on the path of nothing.



Each node is labelled with the container name you pass to **docker exec**, and each end of a link with the address on that interface. The roles:

Machine	Address	What it does
107_L7_LAB_resolver	107.1.0.10	caching resolver for the campus; the machine under attack
107_L7_LAB_victim	107.1.0.20	campus host; asks the resolver and believes it
107_L7_LAB_auth	107.2.0.10	authoritative for uni.lab ; the real www.uni.lab
107_L7_LAB_attacker	107.3.0.10	authoritative for attack.lab ; where you work
107_L7_LAB_router	107.1.0.1	campus side; also 107.2.0.1 and 107.3.0.1, forwarding between all three networks

The resolver, the authoritative server and the attacker's machine all run **named**, from BIND 9.18. Three things about the setup are deliberate and you should know all three before you start, because two of them are what make the attack possible and the third is what makes it observable in one sitting:

- **The resolver arrives weakened.** Its outgoing queries always leave from port 33333 instead of a fresh port each time, it will look a name up for any client that asks from any network, and it does not check signatures. Those three settings live in `/etc/bind/hardening.conf` on the resolver, and Part 2 is where you rewrite them. BIND warns on every configuration load that specifying a query source port is not recommended; that warning is the point.
- **The router does not filter on source address.** It forwards a packet on the destination address and checks nothing about where the packet claims to come from, which is what lets a forged source address cross it. Ingress filtering at the network edge is the subject of the source-spoofing lab and is not one of the three defences here.
- **The authoritative server is 400 ms away.** Its replies are held back by that much before they reach the campus, reproducing an authoritative server on another continent or one slow enough under load to take that long. Between two containers on one laptop the real figure is a fraction of a millisecond, which would leave no attack to look at. You will measure the 400 ms yourself in step 1 and every piece of arithmetic in Part 1 uses the figure you measured.

Definition: `docker exec`

Runs one command inside a container that is already running. `-i` keeps your keyboard attached to that command and `-t` gives it a terminal; written together as `-it`, they hand you an interactive shell rather than running one command and exiting. The name after the flags picks the container; the rest of the line is the command to run inside it, here `bash`.

You will need three shells over the course of the lab, opened with `docker exec` from your own terminal. Every command block below names which one it belongs in:

```
# your own terminal: the three shells this lab uses
docker exec -it 107_L7_LAB_attacker bash      # where you attack from
docker exec -it 107_L7_LAB_victim  bash      # the campus host, to see what it is
told
docker exec -it 107_L7_LAB_resolver bash      # the resolver, which you configure in
Part 2
```

The attacker's machine carries `dig` for asking questions, `tcpdump` for reading what arrives on its own interface, `curl` for fetching a page, and one purpose-built tool, `dns-spoof`, which sends forged answers and takes no default addresses, ports or names.

Assessment. This lab is graded on eighteen questions, marked inline below as **Question *N***, each placed at the step whose output it asks about. Answer all eighteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step.

Part 1: the attack

Definition: Zone, and time to live (TTL)

A *zone* is a connected region of the name hierarchy administered as one unit and served by one or more authoritative servers, named after the point in the hierarchy it starts at: `uni.lab` is a zone, and `www.uni.lab` is one name in it. Every record carries a *time to live*, a number of seconds, which is the resolver's instruction on how long it may keep the record cached before asking again. `dig` prints it as the second field of an answer line, and on a cached answer it prints the seconds remaining, counting down. The record's owner sets the value; nothing about it is verified by whoever caches it.

Read the network before touching anything

1. **Ask as the campus host does.** On the campus host, look the name up twice, a few seconds apart. `dig` prints a header, then the question, then the answer section:

```
# 107_L7_LAB_victim: the campus host
dig www.uni.lab
sleep 5
dig www.uni.lab
```

Read three things off each run: the address in the answer line, the **TTL** in the second column of that line, and the `;; Query time:` line at the foot, which is how long the resolver took to answer this particular question.

Question 1. Record the address, the TTL and the query time from both runs. The two query times differ by more than two orders of magnitude. Explain what the resolver did on the first run that it did not do on the second, and state which of the two figures is the one an attacker has to work inside.

2. **Confirm which machine that address belongs to.** The address came out of the resolver's cache, and the record itself carries no evidence of which machine it belongs to. Each of the two machines this name can resolve to in this lab runs a web server on port 80 that returns one line naming itself, so fetching the page tells you which of the two you reached:

```
# 107_L7_LAB_victim
curl http://www.uni.lab/
```

3. **Measure the distance to the authoritative server.** Ask the authoritative server the question directly, going round the resolver, so the query time you read is the round trip to that server and nothing else:

```
# 107_L7_LAB_victim
dig @107.2.0.10 www.uni.lab
```

What the resolver checks before it believes an answer

Definition: Query matching, and the transaction ID

When a resolver sends a query it keeps the query outstanding and waits for a UDP datagram to answer it. RFC 5452 requires it to check six things about an arriving datagram before treating it as that answer, numbered here in the order RFC 5452 section 9.1 lists them:

1. the source address, against the address the query was sent to;
2. the destination address, against the address the query was sent from;
3. the destination port, against the source port the query left from;
4. the *query ID*;
5. the query name;
6. the query class and type.

Anything that mismatches makes the datagram invalid. The query ID, also called the transaction ID, is the first field of every DNS message: RFC 1035 defines it as "a 16 bit identifier assigned by the program that generates any kind of query", copied unchanged into the reply so the requester can "match up replies to outstanding queries". It is a correlation handle, not a password, and 16 bits is 65,536 values.

Definition: Off-path attacker

An attacker that cannot see the traffic it is attacking. Here the resolver's queries to the authoritative server cross the campus link and the authoritative server's link, and never the attacker's, so no capture the attacker runs on its own interface will ever contain one. Everything an off-path attacker knows about that exchange it either worked out in advance or made the resolver reveal some other way; anything left over it has to guess.

An **off-path attacker** forging an answer has to satisfy all six checks at once, and has to do it without ever reading the query it is answering. Some of the six are fixed by the attacker's own choices: it decides which name to answer for, and the addresses in the exchange follow from which server is authoritative for that name, which is public. The rest are values the resolver picked, and the attacker either measures them or guesses them. The next two steps are the measurement.

Question 2. Using the **query matching** rules, take the position of a machine on 107.3.0.0/24 forging an answer for **www.uni.lab**. Go through all six attributes and say, for each, whether the attacker can satisfy it without ever seeing the resolver's query, and how. Then state how many values remain to be guessed if the resolver's source port is known, and how many if it is not.

Definition: Recursion, and an open resolver

Recursion is the work a resolver does on a client's behalf: chasing the answer to a name it has no record for and returning the finished result, rather than telling the client to go and ask somewhere else. A resolver names the clients it will do that work for as a list of addresses and prefixes; BIND's setting is **allow-recursion**, and on this lab's resolver it currently reads **any**. A resolver that does the work for any client on any network is an *open resolver*.

Definition: Response code (rcode)

A four-bit field in the header of every DNS reply, saying what the name server did with the question. RFC 1035 fixes the values: 0, no error condition; 2, server failure; 3, name error, meaning the name does not exist; and 5, refused, which the RFC attaches to policy, "for example, a name server may not wish to provide the information to the particular requester". `dig` prints the code by name on its `;; ->HEADER<-` line, as `status: NOERROR`, `status: SERVFAIL`, `status: NXDOMAIN` or `status: REFUSED`. The code reports what the server did; it says nothing about whether the records that came back are the ones the zone's operator published.

4. **Find out whether the resolver will work for you.** From the attacker's machine, aimed at the resolver, ask for a name in the zone the attacker itself is authoritative for. If the answer comes back, this resolver performs **recursion** for the attacker's network:

```
# 107_L7_LAB_attacker: where you work from here on
dig @107.1.0.10 probe.attack.lab
```

Question 3. State the **rcode** in the header of that reply and the address it returned. The attacker is authoritative for `attack.lab` and could have answered this question itself without involving anybody. Explain what asking the resolver instead gave the attacker that asking itself would not have, and name the one thing about the resolver's behaviour the attacker can now control.

5. **Read the resolver's outgoing query off your own interface.** The resolver has to come to the attacker's machine to resolve a name in `attack.lab`, and when it does, that query arrives on the attacker's own interface where it can be captured. Start a capture in one shell:

```
# 107_L7_LAB_attacker: shell 1, leave this running
tcpdump -n -i 107-ext 'udp port 53'
```

Definition: tcpdump on a DNS query

`tcpdump` prints one line per packet crossing an interface. `-i` picks the interface, `-n` leaves addresses and ports as numbers instead of looking up names, and the quoted expression is a filter, so only packets matching it are printed. A DNS line reads `<time> IP <source>.<port> > <destination>.<port>: <id><flags> [Nau] <type>? <name>. (<length>)` The number after the last dot of the source address is the source port, and the number immediately after the colon is the transaction ID from the DNS header. The figure in brackets at the end is the length of the DNS message in bytes, not of the datagram carrying it. The characters printed straight after that number, with no space, are header flags rather than part of it: `%` means the checking-disabled bit is set, `+` means recursion desired. `[1au]` counts the records in the additional section, which is one here because the query carries an EDNS0 option record: an extra record a sender adds to advertise the largest UDP reply it will accept and to carry protocol extension flags, one of which is what `dig +dnssec` sets in Stage 3 to ask for signatures. The port and the transaction ID are the two fields this step is being run to read.

In a second shell on the same machine, make the resolver look up three names it has never

seen. Each one is a fresh question, so each one produces a fresh query on the wire:

```
# 107_L7_LAB_attacker: shell 2
dig @107.1.0.10 probe1.attack.lab
dig @107.1.0.10 probe2.attack.lab
dig @107.1.0.10 probe3.attack.lab
```

Look at the three lines in **tcpdump** where 107.1.0.10 is the source, and read the source port and the transaction ID off each. Stop the capture with Ctrl-C when you have them.

Question 4. Give the three source ports and the three transaction IDs. State how many combinations of the two an attacker who has taken this measurement still has to work through, and how many it would face if the field that repeated across the three queries varied as widely as the field that did not. Show the arithmetic for both.

Winning the race

Definition: The race window

The interval between the resolver sending its query and the real answer arriving. Inside it the resolver is waiting and will accept the first datagram that matches; once the real answer lands the question is answered, the outstanding query is gone, and every later datagram carrying that same transaction ID is discarded whatever it says. So the width of the window, multiplied by the rate an attacker can send at, is the number of guesses one attempt is worth.

The attacker's tool sends one forged answer per transaction ID, working through all 65,536 of them and starting again, for as long as it is told to run. Every value it needs is named on the command line: the address to forge, the resolver to send to, the port that resolver is listening for its answer on, the question to answer, the address to answer with, and the TTL to claim.

6. **Make sure the name is not already cached.** A resolver holding a valid cached answer sends no query, and with no query there is no **race window**. Watch the TTL fall to nothing:

```
# 107_L7_LAB_victim: repeat until the answer disappears from the cache
dig +short www.uni.lab
```

A real off-path attacker has to wait this out, and the wait is the TTL, so a long TTL is what costs an attacker time: at this zone's 60 seconds the next window is never more than a minute away. In this lab you also hold the resolver, so you can empty its cache instead of waiting, which is worth knowing now because Part 2 has you re-run the attack several times:

```
# 107_L7_LAB_resolver: empties the whole cache
rndc flush
```

7. **Start the flood, then ask the question.** In the attacker's first shell, start sending forged answers and leave it running. The port is the one you measured, and the TTL is the attacker's own choice:

```
# 107_L7_LAB_attacker: shell 1, leave this running for its full 20 seconds
dns-spoof --from 107.2.0.10 --to 107.1.0.10 --dport 33333 --qname www.uni.lab
--answer 107.3.0.10 --ttl 86400 --seconds 20
```

While it is running, use the second shell to make the resolver ask the question you are answering:

```
# 107_L7_LAB_attacker: shell 2, while the flood is running
dig @107.1.0.10 www.uni.lab
```

When `dns-spoof` finishes it prints how many answers it sent, the rate in packets per second it achieved, and how long one complete sweep of the transaction-ID space took on this machine.

Question 5. Record the rate and the single-sweep time `dns-spoof` reported, and the window you measured in step 3. Calculate how many complete sweeps of the transaction-ID space fit inside that window. Then state what that number means for whether this attack is a gamble or a certainty, and what it would have to fall to before the attack became a gamble again.

8. **Check what the campus now believes.** Nothing about the campus host has been touched, and nothing about the authoritative server has been touched. Ask from the campus host exactly as in step 1, then follow the address:

```
# 107_L7_LAB_victim
dig www.uni.lab
curl http://www.uni.lab/
```

Question 6. Record the address, the TTL and the banner the campus host now gets. The TTL is not the one you saw in step 1. State where the new value came from, say what if anything checked it before the resolver stored it, and calculate how long the campus will keep being given this answer if nothing intervenes.

Question 7. The real answer from 107.2.0.10 arrived about 400 ms after the forged one, carrying the correct address, from the address the resolver had actually queried. Explain, from the definition of the race window, why it did not replace what the resolver had already stored. Then say what an observer watching only the campus host's traffic could have noticed at the moment the poisoning happened.

Question 8. The attacker chose a TTL of 86,400 seconds. Suppose it had chosen 60 instead, matching the zone's own value. State one thing that becomes harder for the attacker and one thing that becomes harder for whoever runs the campus network, and say which of the two choices you would make as the attacker and why.

Part 2: the defence

Three settings on the resolver, closing three different parts of the attack. The first takes away the one value the attacker measured rather than guessed, so that the guess it has to make is roughly 64,000 times larger. The second takes away the attacker's ability to decide when the

resolver sends a query at all, so it can no longer choose the moment to attack. The third stops the resolver believing any answer for this zone that does not carry a signature it can verify, which is the only one of the three that is still worth anything against an attacker who wins the race anyway.

You will apply them one at a time, from the resolver's shell, by rewriting `/etc/bind/hardening.conf` and restarting `named`. Each rewrite replaces the whole file, so what you type is the complete set of settings in force, and `named` reads it again when it starts:

```
# 107_L7_LAB_resolver: the file you will be rewriting, as it arrives
cat /etc/bind/hardening.conf
```

Restarting `named` also empties the cache, so each stage starts from nothing cached and you do not need a separate flush between experiments.

Stage 1: a fresh source port for every query

Definition: Source-port randomisation

Choosing the source port of each outgoing query at random instead of always using the same one. RFC 5452 requires it: a resolver **MUST** "use an unpredictable source port for outgoing queries from the range of available ports (53, or 1024 and above) that is as large as possible and practicable", and must use different ports for queries that are outstanding at the same time. The same document puts the effect at a factor of 64,000 on the size of the problem an attacker faces, because the port stops being something the attacker can measure once and reuse. BIND does this by default; the setting in this lab's `hardening.conf` is what turns it off.

9. **Delete the fixed port.** **Source-port randomisation** is what BIND does when nothing tells it otherwise, so the change is a deletion. Write the file back without the `query-source` line, then restart:

```
# 107_L7_LAB_resolver
cat > /etc/bind/hardening.conf <<'EOF'
allow-recursion { any; };
dnssec-validation no;
EOF
kill -x named; sleep 1; named -c /etc/bind/named.conf
```

10. **Take the same measurement again.** Repeat step 5 exactly: a capture in one shell on the attacker, three fresh names in another:

```
# 107_L7_LAB_attacker: shell 1
tcpdump -n -i 107-ext 'udp port 53'
```

```
# 107_L7_LAB_attacker: shell 2
dig @107.1.0.10 probe4.attack.lab
dig @107.1.0.10 probe5.attack.lab
dig @107.1.0.10 probe6.attack.lab
```

Question 9. Give the three source ports this time. Using the packets-per-second rate you measured in Question 5, calculate how long it would take to send one forged answer for every combination of source port and transaction ID, taking the port range as the 64,512 ports above 1023. Compare that figure with the race window you measured, and give the fraction of the combinations that fit inside one window.

11. **Run the same attack against it.** Predict the outcome before you run anything, then run the identical command from Part 1, still aimed at port 33333, and trigger the query as before:

```
# 107_L7_LAB_attacker: shell 1, leave running
dns-spoof --from 107.2.0.10 --to 107.1.0.10 --dport 33333 --qname www.uni.lab
--answer 107.3.0.10 --ttl 86400 --seconds 20
```

```
# 107_L7_LAB_attacker: shell 2, while the flood is running
dig @107.1.0.10 www.uni.lab
```

```
# 107_L7_LAB_victim
dig www.uni.lab
```

Question 10. State the prediction you made before running it and what the campus host actually got. Then explain which of the six matching rules the forged answers are now failing, and say whether the resolver looked at anything else about them before discarding them.

Stage 2: recursion for the campus only

The first defence left the attacker able to make the resolver send a query whenever it wanted. That is worth something even to an attacker who has to guess a port: it decides the moment of the attempt, it makes each attempt cost nothing, and it is how the attacker measured the port in the first place.

12. **Name the clients this resolver works for.** `allow-recursion` takes a list of addresses and prefixes. Narrow it to the campus and the resolver's own loopback:

```
# 107_L7_LAB_resolver
cat > /etc/bind/hardening.conf <<'EOF'
allow-recursion { 107.1.0.0/24; localhost; };
dnssec-validation no;
EOF
kill -x named; sleep 1; named -c /etc/bind/named.conf
```

13. **Ask from both sides.** The same question, from the attacker and from the campus host:

```
# 107_L7_LAB_attacker
dig @107.1.0.10 www.uni.lab
dig @107.1.0.10 probe7.attack.lab
```

```
# 107_L7_LAB_victim
dig www.uni.lab
```

Question 11. Give the rcode each of the three lookups returned. Then explain what an off-path attacker would now have to do to get the resolver to send a query for `www.uni.lab` at all, and say whether this setting has made the attack impossible or only more expensive. Be specific about which part of the attack it removes and which part it leaves untouched.

Question 12. Compare this defence with the one in Stage 1. State what each one takes away from the attacker, then say which of the two would still be worth having against an attacker that had got a machine onto `107.1.0.0/24`, and why the other one would not.

Stage 3: check the zone's signatures

Neither defence so far looks at what an answer says. Both make it harder for a forged answer to be accepted as the reply to a query; neither would notice a wrong address arriving through a channel it was willing to accept. The third defence is the one that does, and it works on the answer rather than on the exchange that carried it.

Definition: Signed zone: DNSKEY and RRSIG

A zone's operator can sign it. Each set of records of one type at one name is signed with the zone's private key, and the signature is published beside the records as an *RRSIG* record; the matching public key is published as a *DNSKEY* record at the top of the zone. Anyone holding the public key can check that the records they were handed are the ones the operator published and that nothing altered them on the way. Signing is a decision the zone's operator makes; whether anybody checks the signatures is a separate decision, made by each resolver for itself. `dig +dnssec` asks for these records and prints them alongside the answer.

14. **Look at the artifact before anything reads it.** `uni.lab` has been signed since the lab started, through every step so far. Ask for the answer and the key, with the signatures:

```
# 107_L7_LAB_victim
dig +dnssec www.uni.lab
dig +dnssec uni.lab DNSKEY
```

Question 13. Record which record types came back for each of the two questions, and quote the `;; flags:` line from the first one. The zone has been signed for the whole of this lab, including while the poisoning in Part 1 succeeded. Explain what the presence of an RRSIG in this reply does and does not tell you about what the resolver checked.

Definition: Trust anchor

The key a validating resolver starts from. RFC 4033 defines it as a configured DNSKEY record, or a hash of one, that the resolver's operator obtains "via some secure or trusted means outside the DNS protocol" and installs by hand. It has to arrive outside DNS because a key delivered inside a DNS answer could have been forged by exactly the attacker it is meant to catch. In this lab `uni.lab`'s operator has published its key to the resolver's operator, and it is sitting on the resolver in `/etc/bind/uni.lab.key`, installed nowhere. Having a trust anchor for a zone is also a statement that the zone is expected to be signed, so an unsigned answer for a name inside it is a failure rather than something to fall back on.

Definition: The AD bit

One bit in the header of a reply, carrying one claim from the resolver to its client: everything in this answer was checked and it verified. RFC 4035 requires a recursive server not to set it "unless the name server considers all RRsets in the Answer and Authority sections of the response to be authentic", and says it should be set if and only if its own validation reached that conclusion. `dig` prints it as `ad` in the `;; flags:` line. It is the resolver's report about work the resolver did, not something the authoritative server said and not a statement that the zone is signed.

15. **Install the key and switch validation on.** Read the [trust anchor](#) you have been given, then put it where `named.conf` already includes it from, and turn checking on:

```
# 107_L7_LAB_resolver
cat /etc/bind/uni.lab.key
cp /etc/bind/uni.lab.key /etc/bind/trust-anchors.conf
cat > /etc/bind/hardening.conf <<'EOF'
allow-recursion { 107.1.0.0/24; localhost; };
dnssec-validation yes;
EOF
pkill -x named; sleep 1; named -c /etc/bind/named.conf
```

```
# 107_L7_LAB_victim
dig www.uni.lab
```

Question 14. Quote the `;; flags:` line from this lookup and compare it with the one you quoted in Question 13. Name the single flag that has appeared. The zone was signed for both lookups and the authoritative server sent the same records both times, so state what changed to make the flag appear. Then explain why installing the key was needed as well as switching validation on rather than instead of it, and what the resolver would have done with this answer if only one of the two had been done.

Definition: Bogus, and SERVFAIL

The verdict a validating resolver reaches about records it believes it should be able to verify and cannot, whether because a signature failed or because the DNSSEC records that should have been there were missing. RFC 4035 notes such a result "may indicate an attack but may also indicate a configuration error or some form of data corruption", so the resolver withholds rather than substituting anything: asked by an ordinary client, it MUST return RCODE 2, server failure, which `dig` prints as `status: SERVFAIL` with no answer section. A defence that works therefore looks on screen like a lab that is broken.

16. **Give the attacker the race back, deliberately.** With a source port chosen fresh for every query, a forged answer matches an outstanding query at about the rate you calculated in Question 9, so nothing would reach the point where a signature is checked. Pin the port again for one experiment, leaving validation on, so that a forged answer can arrive first and you can see what happens to it:

```
# 107_L7_LAB_resolver: putting a weakness back, on purpose, for one measurement
cat > /etc/bind/hardening.conf <<'EOF'
query-source address * port 33333;
allow-recursion { 107.1.0.0/24; localhost; };
dnssec-validation yes;
EOF
pkill -x named; sleep 1; named -c /etc/bind/named.conf
```

Recursion is still restricted to the campus, so the attacker can no longer trigger the query itself. The campus host is the trigger now, which is how an off-path attacker works once Stage 2 is in place: it floods and waits for somebody legitimate to ask.

```
# 107_L7_LAB_attacker: shell 1, leave running
dns-spoof --from 107.2.0.10 --to 107.1.0.10 --dport 33333 --qname www.uni.lab
--answer 107.3.0.10 --ttl 86400 --seconds 20
```

```
# 107_L7_LAB_victim: while the flood is running
dig www.uni.lab
curl http://www.uni.lab/
```

Then wait for the flood to finish and ask once more:

```
# 107_L7_LAB_victim: after dns-spoof has stopped
dig www.uni.lab
```

Question 15. Give the rcode and the answer section the campus host got during the flood, and the rcode, address and flags it got afterwards. The forged answer won the race in both this run and the one in Part 1, and the outcomes differ. Explain what the resolver did with the forged answer this time, and account for the difference between what the campus host lost here and what it lost in Part 1.

Question 16. The real answer was arriving from 107.2.0.10 throughout the flood, 400 ms behind every forged one. Working from what you recorded in Question 15, explain why the

resolver did not hand the campus host that answer instead. Then state what a validating resolver can and cannot do for the campus while an attacker is winning the race, and name the class of attack the campus is exposed to for as long as the flood lasts.

17. **Put the resolver back the way it should be.** The pinned port was for one measurement. Write the hardened set of settings and restart, then confirm the campus is unaffected:

```
# 107_L7_LAB_resolver
cat > /etc/bind/hardening.conf <<'EOF'
allow-recursion { 107.1.0.0/24; localhost; };
dnssec-validation yes;
EOF
pkill -x named; sleep 1; named -c /etc/bind/named.conf
```

```
# 107_L7_LAB_victim
dig www.uni.lab
curl http://www.uni.lab/
```

Comparing the three

Definition: DNS cookies

A fourth mechanism against forged answers, which this lab leaves switched off on both name servers. Specified in RFC 7873, cookies have the resolver put a value of its own in each query, which the server echoes back in its reply; the resolver then discards any reply that does not carry the right one. That adds one more value a forged answer has to carry, alongside the six attributes of the matching rule. Cookies are switched off on the resolver and on the authoritative server, which is why an answer in this lab is matched on the addresses, the ports, the transaction ID and the question alone.

The last two questions are about all four mechanisms together, the three you applied and **DNS cookies**.

Question 17. You have now applied three defences. For each one, state in a sentence what it changes about the attack: which of them makes the attack improbable, which changes who can start it, and which makes it ineffective even when it succeeds. Then rank them by what you would deploy first on a campus resolver, and justify the order in terms of what an attacker can still do after each step.

Question 18. Consider an attacker that has moved onto 107.1.0.0/24 and can capture the resolver's outgoing queries. Go through the four mechanisms named in this lab, the three you applied and DNS cookies, and state for each whether it still stops that attacker and why. Then name the one property such an attacker has that none of the other defences can take away, and say which mechanism is unaffected by it.

Checks: what “done” means

- **Part 1:** the campus host resolves `www.uni.lab` to 107.3.0.10 and `curl` reaches the impostor, with a TTL you chose, and you can say how many transaction IDs the flood covered inside the window.

- **Stage 1:** three consecutive probes leave from three different source ports, and the same attack leaves the campus host resolving 107.2.0.10.
- **Stage 2:** the attacker's query returns REFUSED and the campus host's returns 107.2.0.10.
- **Stage 3:** a clean lookup carries `ad` in its flags, and a lookup made while a forged answer is winning the race returns SERVFAIL rather than the attacker's address.

`scripts/status.sh` prints which of the three settings is in force, which address the campus is currently given for the name, and which machine replies there, so you can read the before and after of each stage without shelling in. `scripts/reset.sh` puts all three settings back to their weak starting state, removes the trust anchor and empties the cache, so any part can be run again from the beginning.

Safety. Everything here lives on one isolated router and switch with no route off the host. Every forged packet, query and capture only ever reaches the lab's own containers, and `dns-spoof` sends only to the address you name on its command line. Sending forged packets to a resolver you do not own or have written permission to test is an offence in most jurisdictions, including under the Crimes Act 1961 in New Zealand, and it is a breach of the acceptable-use policy of every university and consumer internet provider.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only undoes your configuration; only **Teardown** deletes the lab.